

Algorithmic Structure of the Stabilized Porous Navier–Stokes Solver

Antigravity AI Project Authors

May 17, 2026

Abstract

This document specifies the nonlinear solver pipeline that drives a single invocation of `run_simulation` in the stabilized porous Navier–Stokes solver. The pipeline is presented as a strict module hierarchy: at the top sits the orchestrator (Algorithm O), which calls two mid-level routines (Algorithms B and C, the Stage I and Stage II solvers), each of which in turn invokes a shared Newton kernel (Algorithm A), which is itself composed of three leaf primitives (Algorithms A.1, A.2, A.3). A graphical map of this hierarchy is given in Section 2. Each algorithm is documented in two parts: a brief *interface card* listing its callers, callees, inputs and outputs; and a procedural pseudocode box.

The presentation is organised in two strict tiers. Part I describes the *core algorithm*. Part II documents two *optional extensions* that augment the core algorithm in specific settings: a plateau verifier based on the Method of Manufactured Solutions, which fires only when a candidate exact solution is supplied; and a parameter-sweep harness that re-tunes a small number of tolerances on a per-case basis. The two tiers are kept strictly separate: passages and algorithm-line additions belonging to the extensions are typeset in [this colour](#) throughout.

Contents

I	The Core Algorithm	2
1	Overview and Algorithmic Challenges	2
2	Module Hierarchy	3
3	Parameter Taxonomy and Code Equivalence	4
3.1	Numerical parameters of the core algorithm	4
3.2	Algorithm–to–code correspondence	4
4	Mathematical Preliminaries	4
4.1	The diagonally equilibrated merit function	4
4.2	The Gauss–Newton directional-derivative identity	5
5	The Top: Algorithm O (Orchestrator)	6
6	The shared cascade: Algorithm B (RobustNonlinearCascade)	7
7	The Newton Kernel: Algorithm A and Its Leaves	8
7.1	Algorithm A: ExactNewtonPipeline	8
7.2	Algorithm A.1: SolveLinearSystem	9
7.3	Algorithm A.2: ArmijoLinesearch	10
7.4	Algorithm A.3: EvaluateSafeguards	11

8	Stage II: Algorithm C (OSGSFractionalRelaxation)	12
8.1	State drift: ℓ^∞ vs continuous L^2	13
8.2	Projection drift in the mass-weighted inner product	14
8.3	Choosing the stopping mode	14
8.4	Bootstrap, mass-matrix pre-factoring, and warmup	14
8.5	Projection policy and the constant- σ trim	14
8.6	Block-wise Anderson acceleration	15
8.7	Failure of the inner cascade	15
9	Convergence-Analysis Notes	15
9.1	The false-equivalency failsafe	15
9.2	Why the OSGS stopping mode matters	16
A	Code–Notation Correspondence	16
II	Optional Extensions	16
B	Extension Hierarchy	16
C	Method of Manufactured Solutions Plateau Verifier	16
C.1	When this extension fires	16
C.2	Numerical versus discretisation convergence	17
C.3	Algorithm D and its interface	17
C.4	How Algorithm D hooks into the core	18
C.5	Parameters of this extension	18
D	Parameter-Sweep Harness Adaptations	18
D.1	When this extension fires	18
D.2	Adaptive τ_{ftol}	18
D.3	Adaptive noise floor	19
D.4	Per-case Picard widening	19
D.5	Parameters of this extension	19

Part I

The Core Algorithm

1 Overview and Algorithmic Challenges

The continuous problem is the stationary, incompressible, generalized porous Navier–Stokes system on a domain $\Omega \subset \mathbb{R}^d$,

$$\begin{aligned} \mathbf{u} \cdot \nabla \mathbf{u} - \nabla \cdot (2\nu \nabla^s \mathbf{u}) + \sigma(\alpha, \mathbf{u})\mathbf{u} + \nabla p &= \mathbf{f}, \\ \nabla \cdot \mathbf{u} &= 0, \end{aligned} \tag{1}$$

discretised by an equal-order Lagrangian pair $(\mathbf{V}_h, Q_h)$ of degree $(k_v, k_p) = (k, k)$ and stabilised by a Variational Multiscale (VMS) formulation. The drag tensor $\sigma(\alpha, \mathbf{u}) = a(\alpha) + b(\alpha)|\mathbf{u}|$ couples the Darcy–Forchheimer regime to the convective momentum balance; the porosity field α varies in space, and the high-Re, high-Da regimes the solver must handle are exactly those in which the system becomes simultaneously convection-dominated, reaction-dominated, and stiff.

Nonlinearity from convection and Forchheimer drag. Newton’s method converges quadratically once the iterate enters its quadratic basin, but locating that basin from an arbitrary initial guess is non-trivial when both $\mathbf{u} \cdot \nabla \mathbf{u}$ and the $|\mathbf{u}|$ factor in σ amplify any error in $\mathbf{u}$. The solver must detect divergence and respond—either by shortening the step (line search), by switching to an iteration map with a wider basin (Picard), or by aborting cleanly.

Implicit OSGS projection. In the Orthogonal Sub-Grid Scale formulation the sub-grid component lives in the orthogonal complement $\mathcal{X}_{h0}^\perp$ of the finite element space; in practice the projection $\pi_h \in \mathcal{X}_h$ of the strong residual is computed, and the sub-grid scale is identified with $(I - \Pi_h)\mathcal{R}$. The projection itself depends on $U_h = (\mathbf{u}_h, p_h)$, so a fully monolithic Jacobian would embed the Fréchet derivative of an L^2 projection operator. Rather than carrying this dense block, the solver freezes π_h , solves the nonlinear primary system, then re-projects, in a staggered (block-coordinate) fixed point.

Architecture of the response. The architecture that emerges is a two-stage cascade. *Stage I* drives the algebraic system to a converged ASGS state, regardless of the final stabilization method requested. *Stage II* either reports the ASGS state directly or, when the user has selected OSGS, runs a staggered outer loop on (U_h, π_h) initialised at the Stage I solution. The outermost orchestration (Algorithm 1) wires all of these together.

ASGS is always run first; OSGS, when requested, refines the ASGS state through a staggered projection loop.

2 Module Hierarchy

The algorithms below are organised in a strict call hierarchy. The orchestrator (Algorithm 1) sits at the top and dispatches to two stage solvers (Algorithms 2 and 7), both of which build on a single shared Newton kernel (Algorithm 3). The Newton kernel itself is built from three leaf primitives (Algorithms 4, 5, 6). Figure 1 shows the full tree.

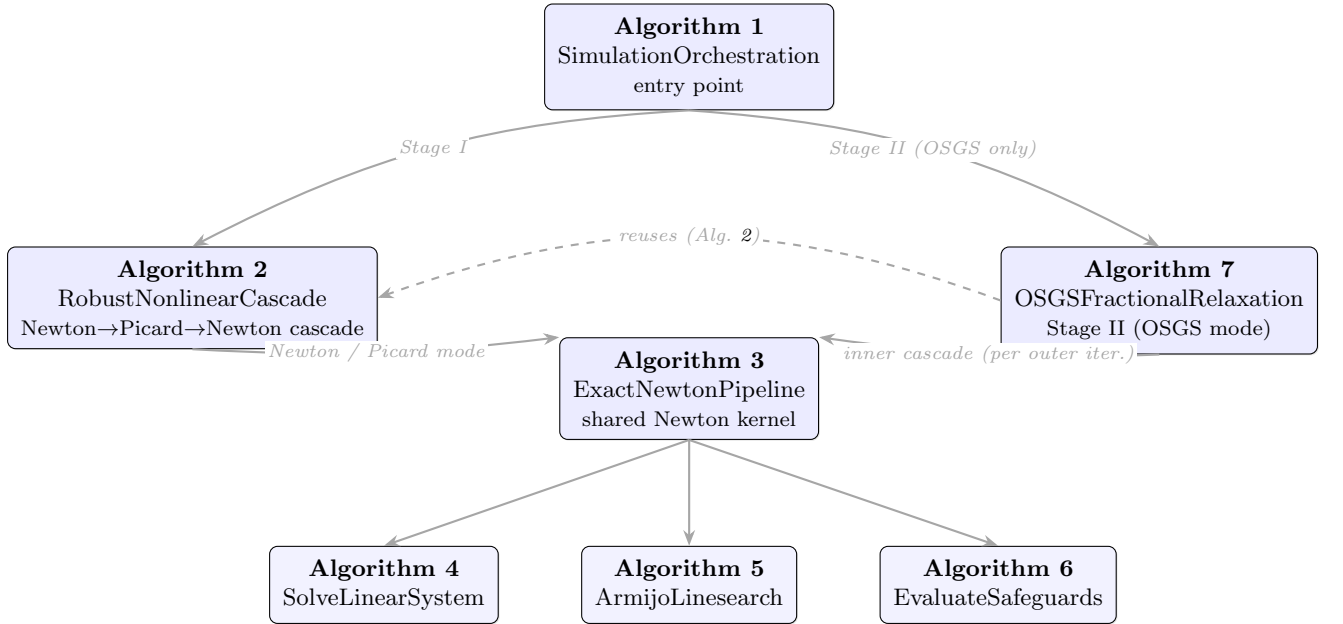


Figure 1: Module hierarchy of the core algorithm. Arrows point from caller to callee. Each edge is annotated with the calling context. Algorithm 3 is the shared workhorse: Algorithm 2 invokes it three times in sequence (Newton, then Picard, then Newton) with different Jacobians, and is itself reused by Algorithm 7 once per staggered outer iteration with the projection π_h frozen (the dashed edge). Leaf primitives A.1–A.3 are pure procedures with no further dependencies.

Reading order. This document presents the modules top-down: first the orchestrator (§5), then the two stage solvers (§6, §8), then the shared Newton kernel and its leaves (§7). Each algorithm is introduced by a short *interface card* that summarises its position in the tree—callers, callees, inputs, outputs—before the procedural box. Mathematical preliminaries shared by several algorithms (the diagonally equilibrated merit function and its directional derivative identity) are gathered in §4.

3 Parameter Taxonomy and Code Equivalence

3.1 Numerical parameters of the core algorithm

Table 1 enumerates every numerical parameter consumed by the algorithms of Part I, grouped by role. Parameters that belong to the optional extensions of Part II are tabulated separately in the corresponding sections.

Table 1: Numerical parameters of the core nonlinear-solver pipeline. Throughout the table, `sol = config.numerical.method.solver` (a flat `SolverConfig @kwdef` struct) and `stab = config.numerical.method.stabilization`. The `SafeNewtonSolver` struct that runs the inner cascade is built from these fields by `SafeNewtonSolver(::LinearSolver, ::SolverConfig)`; its post-construction field names (e.g. `c1`, `max_iters`) are shorter aliases of the config field names. Code-side references in this table are written against the config struct, not the built solver.

Symbol	Field in code	Role / effect
<i>Inner Newton iteration controls</i>		
N_N	<code>sol.newton.iterations</code>	Cap on inner Newton iterations.
N_P	<code>sol.picard.iterations</code>	Cap on Picard fallback iterations.
N_{inc}	<code>sol.max_increases</code>	Consecutive merit-function expansions tolerated before declaring divergence.
τ_{ftol}	<code>sol.ftol</code>	Terminal residual tolerance in ℓ^∞ ; the dominant convergence criterion.
τ_{xtol}	<code>sol.xtol</code>	Lower bound on the accepted step norm $\ \alpha\delta\mathbf{x}\ $ before declaring coordinate stagnation.
τ_{stag}	<code>sol.stagnation_noise_floor</code>	Noise-floor residual; a non-converged exit below this may still be classified as success (Sec. 9).
α_{min}	<code>sol.linesearch.alpha_min</code>	Smallest line-search step before giving up.
c_1	<code>sol.armijo_c1</code>	Armijo / monotone-residual descent constant.
γ	<code>sol.linesearch.contraction_factor</code>	Backtracking factor: $\alpha \leftarrow \gamma\alpha$.
N_{ls}	<code>sol.max_linesearch.iterations</code>	Bound on line-search inner iterations.
F_{div}	<code>sol.divergence_merit_factor</code>	Threshold by which the merit must expand to count as one “increase”.
m	(built into the <code>SafeNewtonSolver</code> as <code>.mode</code>)	Cascade mode: <code>:newton</code> (merit-Armijo) or <code>:picard</code> (monotone residual reduction); cf. §4.2 and Alg. 5.
<code>freeze_cusp</code>	<code>sol.freeze_jacobian_cusp</code>	Boolean: skip Jacobian recomputation at Forchheimer cusps to avoid singular updates.
<i>OSGS staggered outer loop</i>		
N_{OSGS}	<code>stab.osgs.iterations</code>	Cap on outer (staggered) iterations.
N_{inner}	<code>stab.osgs.inner_newton_iters</code>	Cap on inner Newton iterations per outer step.
τ_{OSGS}	<code>stab.osgs.tolerance</code>	Tolerance on the state drift d_{state}^m .
τ_{proj}	<code>stab.osgs.projection_tolerance</code>	Tolerance on the mass-weighted projection drift d_π^m .
m_w	<code>stab.osgs.warmup.iterations</code>	Outer iterations during which the inner Newton uses a relaxed tolerance.
τ_w	<code>stab.osgs.warmup.tolerance</code>	Relaxed inner tolerance during warmup.
$\text{Mode}_{\text{drift}}$	<code>stab.osgs.state_drift.scale</code>	Drift metric: <code>Linf</code> or <code>L2.mass</code> . See Sec. 8.1.
$\text{Mode}_{\text{stop}}$	<code>stab.osgs.stopping_mode</code>	Convergence predicate: <code>state_drift</code> , <code>projection_drift</code> , or <code>both</code> .
m, β	<code>sol.accelerator.{m,relaxation_factor}</code>	Anderson history depth and damping.

3.2 Algorithm-to-code correspondence

4 Mathematical Preliminaries

Two mathematical objects are referenced repeatedly by several algorithm boxes below. We define them once here so that the boxes themselves remain procedural.

4.1 The diagonally equilibrated merit function

Let $\mathbf{b}(\mathbf{x}) \in \mathbb{R}^N$ be the assembled residual of the (possibly stabilised) discrete weak form, and $\mathcal{J}(\mathbf{x}) = \partial\mathbf{b}/\partial\mathbf{x}$ its Jacobian. The line-search merit function used throughout the cascade is the diagonally equilibrated quadratic

$$\Phi_w(\mathbf{x}) = \frac{1}{2} \sum_{k=1}^N \left(\frac{b_k(\mathbf{x})}{w_k} \right)^2. \quad (2)$$

Table 2: Algorithm $\leftrightarrow$ code signature mapping (core algorithm).

Algorithm	Code entry point
SimulationOrchestration (Alg. 1)	<code>run_simulation(path) $\rightarrow$ solve_system(setup, form, solvers, cfg, x0).</code>
RobustNonlinearCascade (Alg. 2)	The Newton $\rightarrow$ Picard $\rightarrow$ Newton cascade inside <code>solve_system</code> ; appears twice—once for Stage I (ASGS operators, $\mathbf{x}_{\text{backup}}^0$) and once inside the OSGS outer-loop body of Algorithm 7 (OSGS operators, $\mathbf{x}_{\text{prev}}$ at the start of each outer iteration).
OSGSFractionalRelaxation (Alg. 7)	The OSGS staggered loop inside <code>solve_system</code> .
ExactNewtonPipeline (Alg. 3)	<code>_safe_solve_inner!</code> (<code>src/solvers/nonlinear.jl</code>), reused with either an exact-Newton or a Picard Jacobian.
SolveLinearSystem (Alg. 4)	<code>eval_linear_system_resolution!(dx, A, b, solver, cache).</code>
ArmijoLinesearch (Alg. 5)	<code>eval_armijo_linesearch_pass!.</code>
EvaluateSafeguards (Alg. 6)	<code>eval_safeguard_termination_bounds!.</code>

The weights w_k are recomputed at every Newton iteration from the diagonal of the current Jacobian. Their definition is mode-dependent: when the discrete trial space is a multi-field FE space with a ConsecutiveMultiFieldStyle DOF ordering (the only case used in this codebase: velocity $\oplus$ pressure), the equilibration is performed *per block*. Let $\{\mathcal{B}_q\}_q$ denote the disjoint per-field DOF index ranges (so $\bigcup_q \mathcal{B}_q = \{1, \dots, N\}$). Then

$$w_k = \begin{cases} \max(|\mathcal{J}_{kk}|, \varepsilon_{\text{mach}} \max_{j \in \mathcal{B}_{q(k)}} |\mathcal{J}_{jj}|, \varepsilon_{\text{mach}}) & \text{multi-field, } k \in \mathcal{B}_{q(k)}, \\ \max(|\mathcal{J}_{kk}|, \varepsilon_{\text{mach}} \|\text{diag}(\mathcal{J})\|_{\infty}, \varepsilon_{\text{mach}}) & \text{single-field fallback.} \end{cases} \quad (3)$$

The first form is the production rule for the saddle-point system at issue; the second is the legacy global rule, retained only for the unrealistic single-field case in which $\mathcal{B}_q$ cannot be detected.

Why equilibrate? The discrete residual mixes momentum-balance rows—measured in physical units of force per unit volume, and possibly amplified by a large Reynolds or Damköhler number—with mass-balance rows, whose natural magnitude is dimensionally different. A raw unweighted merit $\frac{1}{2}\|\mathbf{b}\|^2$ would let the momentum rows dominate entirely, so the line search would be quietly indifferent to mass-conservation errors. Equilibrating by w_k rescales every row to its own natural unit. The lower-bound floor $\varepsilon_{\text{mach}} \max_{j \in \mathcal{B}_{q(k)}} |\mathcal{J}_{jj}|$ prevents a genuinely small diagonal entry from triggering division by zero.

Why per block? A global $\|\text{diag}(\mathcal{J})\|_{\infty}$ in the floor would be dominated by the momentum-block diagonal whenever the Reynolds or Darcy regime is severe; the pressure-block floor would then be so close to $\varepsilon_{\text{mach}}$ (momentum scale) that mass-residual rows would still be effectively un-weighted relative to momentum-residual rows, defeating the equilibration entirely. The per-block floor scales each field’s residual against *its own* diagonal magnitudes, which is the only choice that gives the line search a faithful view of mass-residual descent.

4.2 The Gauss–Newton directional-derivative identity

For a Newton step $\delta\mathbf{x}$ satisfying $\mathcal{J}\delta\mathbf{x} = \mathbf{b}$, the directional derivative of $\Phi_{\mathbf{w}}$ along $-\delta\mathbf{x}$ at $\mathbf{x}$ is

$$D = -\sum_k w_k^{-2} b_k (\mathcal{J}\delta\mathbf{x})_k = -\sum_k w_k^{-2} b_k^2 = -2\Phi_{\mathbf{w}}(\mathbf{x}). \quad (4)$$

The Newton-mode Armijo predicate of Algorithm 5 therefore reduces to $\hat{\Phi} \leq \Phi(1 - 2c_1\alpha)$, which requires no extra gradient evaluation.

Picard mode invalidates the identity. The identity (4) holds *only* because the search direction is the exact Newton step. The Picard Jacobian drops the linearised convective cross-term and the linearised Forchheimer term (cf. §6), so the resulting $\delta\mathbf{x}$ does not satisfy $\mathcal{J}\delta\mathbf{x} = \mathbf{b}$ for the true $\mathcal{J}$; the cancellation in the second equality above breaks and D is in general not -2Φ . Forcing the merit-based Armijo predicate on a Picard step would either reject descent directions for the wrong reason or accept ascent directions spuriously. The implementation therefore branches the acceptance test by solver mode: Newton uses the

merit-based Armijo above; Picard uses the monotone residual reduction $\|\hat{\mathbf{b}}\|_\infty \leq (1 - c_1\alpha)\|\mathbf{b}\|_\infty$, which is mode-agnostic because it depends only on the residuals at the two iterates. This branching is shown explicitly inside Algorithm 5.

5 The Top: Algorithm O (Orchestrator)

The orchestration drives one simulation end-to-end. Its central organising principle is a two-stage cascade: an unconditional ASGS algebraic initialisation followed by a method-dependent dispatch. Stage I runs even when the user has requested OSGS, because the staggered projection loop is much better conditioned when initialised at a converged ASGS state than at a zero initial guess.

Called by: (top-level entry point; not called from elsewhere)
Calls: Algorithm 2 (Stage I); Algorithm 7 (Stage II, OSGS mode only)
Inputs: Configuration Θ (validated JSON)
Outputs: Final fields (u_h, p_h, π_h) ; success flag S

Algorithm 1: SimulationOrchestration: end-to-end driver.

Input: Configuration Θ .
Output: (u_h, p_h, π_h) , success flag S .

```

build mesh  $M$ , FE pair  $(X, Y)$ , unconstrained mirrors  $(V_{\text{free}}, Q_{\text{free}})$ , formulation  $F$ , topology  $T$ ,
initial guess  $\mathbf{x}^0$ ;

/* Stage I. Algebraic initialisation in ASGS mode ( $\pi_h \equiv 0$ ). */
( $S, U_h^{\text{ASGS}}$ )  $\leftarrow$  RobustNonlinearCascade( $\mathbf{x}^0, \mathbf{x}_{\text{backup}}^0, \text{op}_N^{\text{ASGS}}, \text{op}_P^{\text{ASGS}}$ ); // Alg. 2
if not  $S$  then
    return failure (no fields written);
end

/* Stage II. Method dispatch. */
if  $\Theta.\text{method} = \text{ASGS}$  then
    if  $\text{mms\_cfg.enabled}$  then
         $I_{\text{pass}} \leftarrow 0$ ; initialise plateau-history  $\mathcal{H}$  at  $U_h^{\text{ASGS}}$ ;
        for cycle = 1, ...,  $K_{\text{extra}}$  do
            run one Newton iteration on top of  $U_h^{\text{ASGS}}$ ;
            ( $S_{\text{plat}}, I_{\text{pass}}$ )  $\leftarrow$  VerifyMMSPlateau(...); // Alg. 8
            if  $S_{\text{plat}}$  then
                break;
            end
        end
    end
    return ( $U_h^{\text{ASGS}}, \pi_h \equiv 0$ );
end

if  $\Theta.\text{method} = \text{OSGS}$  then
    pre-assemble and LU-factor  $M_u = (V_{\text{free}}, U_{\text{free}})_{L^2}, M_p = (Q_{\text{free}}, P_{\text{free}})_{L^2}$  once;
     $\pi_h^0 \leftarrow 0$ ; // deliberate zero bootstrap; see §8.4
    if  $\text{mms\_cfg.enabled}$  then
         $N_{\text{OSGS}}^{\text{eff}} \leftarrow N_{\text{OSGS}} + K_{\text{extra}}$ ; // inflate outer-loop budget for plateau
        verification
    end
    ( $S, \pi_h$ )  $\leftarrow$  OSGSFractionalRelaxation( $U_h^{\text{ASGS}}, \pi_h^0$ ); // Alg. 7; MMS hook in Alg. 7
    fires when  $\text{mms\_cfg.enabled}$ 
end

export_results(VTK | HDF5);

```

Step by step. The construction lines build the mesh, the finite-element pair, and the unconstrained projection mirrors. The unconstrained spaces $V_{\text{free}}, Q_{\text{free}}$ are required only by the OSGS projection step (§8); they carry no Dirichlet data, because projecting onto a Dirichlet-constrained space introduces an

$O(1)$ boundary residual that destroys the expected $O(h^{k+1})$ FE convergence rate.

Stage I. A single call to RobustNonlinearCascade (Algorithm 2) composes an exact-Newton attempt, a Picard globalisation pass, and a final Newton polish, all with $\pi_h \equiv 0$. The output U_h^{ASGS} is the algebraic root of the ASGS system; when Stage II is OSGS, the staggered loop descends from this state with an already-converged primary field. The same cascade module is reused inside Stage II (Algorithm 7) with the OSGS operators and a per-outer-iteration restore vector.

Stage II. Either accept the ASGS state, or hand off to the OSGS outer loop. The mass matrices M_u, M_p are pre-assembled and LU-factored exactly once before the OSGS loop begins. Because the projection target spaces never change during the loop, these factorisations are reused at every outer iteration, which converts every L^2 projection into a constant-cost forward/back substitution.

Failure propagation. If Stage I returns $S = \text{False}$, the orchestrator returns immediately without writing output. The diagnostics cache carries the algebraic exit reason so the calling script can distinguish a divergent run from a converged one.

6 The shared cascade: Algorithm B (RobustNonlinearCascade)

When the safeguarded Newton loop terminates without converging, the orchestration does not give up: it transitions to a Picard iteration map and then re-engages Newton. Algorithm 2 packages this Newton $\rightarrow$ Picard $\rightarrow$ Newton cascade as a single *generic* module that does not depend on which stabilisation mode it is driving. It is therefore reused as-is by both Stage I (called from Algorithm 1 with $\pi_h \equiv 0$ and exact-Newton and Picard ASGS operators) and the inner solve of Stage II (called from Algorithm 7 once per staggered outer iteration with π_h frozen at the previous projection and OSGS-mode operators).

The only behavioural axis that varies between the two callers is the *restore vector* used when Picard's first attempt has to be reset. Stage I restores to a copy of the original initial guess $\mathbf{x}_{\text{backup}}^0$ taken at the very start of the simulation; Stage II restores to the iterate at the start of the current outer staggered iteration $\mathbf{x}_{\text{prev}}$. The algorithm box below takes the restore vector as an explicit parameter, so that both callers reuse a single module without caller-specific branching.

Called by: Algorithm 1 (Stage I); Algorithm 7 (OSGS inner solve, per outer staggered iteration)
Calls: Algorithm 3 (invoked up to three times in sequence, with different Jacobians)
Inputs: Initial guess $\mathbf{x}^0$; restore vector $\mathbf{x}_{\text{restore}}$; exact-Newton and Picard nonlinear operators op_N, op_P
Outputs: Success flag S and the converged or best-seen iterate $\mathbf{x}^*$

Algorithm 2: RobustNonlinearCascade: the Newton $\rightarrow$ Picard $\rightarrow$ Newton cascade. Stage I instantiates $\mathbf{x}_{\text{restore}}$ with a backup copy of the initial guess; Algorithm 7 instantiates it with the iterate at the start of the current outer staggered iteration.

Parameters: $N_N, N_P, \tau_{\text{ftol}}, \tau_{\text{stag}}$ and all Newton/line-search parameters from Table 1.

Input: Initial guess $\mathbf{x}^0$; restore vector $\mathbf{x}_{\text{restore}}$; exact-Newton operator op_N ; Picard operator op_P .

Output: $(S, \mathbf{x}^*)$.

```

( $\mathbf{x}^*, S_N, \dots$ )  $\leftarrow$  ExactNewtonPipeline( $\mathbf{x}^0, \text{op}_N$ );
if  $S_N = \text{True}$  then
  | return (True,  $\mathbf{x}^*$ ) ;                               // Newton sufficed; no fallback
end
restore  $\mathbf{x} \leftarrow \mathbf{x}_{\text{restore}}$ ;
( $\mathbf{x}^*, S_P, \dots$ )  $\leftarrow$  ExactNewtonPipeline( $\mathbf{x}, \text{op}_P$ );
if any component of  $\mathbf{x}^*$  is non-finite then
  | return (False,  $\mathbf{x}_{\text{restore}}$ ) ;                       // catastrophic state
end
if  $S_P = \text{True}$  then
  | return (True,  $\mathbf{x}^*$ );
end
/* Re-engage Newton from wherever Picard left the iterate.                                     */
( $\mathbf{x}^*, S_{N2}, \dots$ )  $\leftarrow$  ExactNewtonPipeline( $\mathbf{x}^*, \text{op}_N$ );
return ( $S_{N2} \vee S_P, \mathbf{x}^*$ );

```

Why Picard works where Newton fails. The Newton Jacobian contains the linearised convective term $\mathbf{u}^k \cdot \nabla \delta \mathbf{u} + \delta \mathbf{u} \cdot \nabla \mathbf{u}^k$ and the linearised Forchheimer term $\partial \sigma / \partial \mathbf{u}$. Both become large in the difficult regimes, and the Jacobian can develop a near-null direction where a small change in $\mathbf{u}$ produces an outsize change in the residual. Newton near such a cusp produces a step that diverges. The Picard Jacobian drops the dependence of $\mathbf{u} \cdot \nabla \mathbf{u}$ and of σ on the increment $\delta \mathbf{u}$, retaining only the “frozen” coefficient form. The resulting iteration map is a contraction in a wider region than Newton’s quadratic basin, at the cost of linear convergence. Picard does not converge to within τ_{ftol} in general—but it does push the iterate into a region where the subsequent Newton retry has a fighting chance.

Triggers. Any non-converged Newton exit triggers the fallback, including `Xtol` stagnation, `IterationCap`, the linear-solver exception path, and any uncaught exception during the Newton call. All of these states leave $\mathbf{x}^0$ outside the basin, and a Picard pass on top costs the same regardless of which symptom triggered the entry.

The catastrophic-state guard. Picard is not bounded in general. After the Picard call, the orchestration checks any(!isfinite, $\mathbf{x}^*$). A NaN or Inf in the iterate is a hard signal that even Picard could not keep the model evaluable, and the fallback restores the supplied $\mathbf{x}_{\text{restore}}$ and returns failure. Without this guard the subsequent Newton call would crash on its first residual evaluation.

7 The Newton Kernel: Algorithm A and Its Leaves

Both Algorithms 2 and 7 call into a single safeguarded Newton routine. Beneath this kernel sit three leaf primitives: a linear-solve helper (A.1), an Armijo backtracking line search (A.2), and a termination-safeguard arbiter (A.3). This section documents the kernel first, then its leaves.

7.1 Algorithm A: ExactNewtonPipeline

Called by: Algorithm 2 (three times per Stage I); Algorithm 7 (once per outer staggered iteration)
Calls: Algorithm 4 (linear solve); Algorithm 5 (Armijo line search); Algorithm 6 (safeguards)
Inputs: Initial fields $\mathbf{x}^0 \in \mathcal{X}_h$; nonlinear operator op (encoding the chosen Jacobian variant)
Outputs: Final iterate $\mathbf{x}^*$; success flag S ; iteration count I ; residual norm $\|\mathbf{b}^*\|_\infty$; exit reason r (Table 3)

Algorithm 3: ExactNewtonPipeline: safeguarded Newton iteration with merit-based line search.

Parameters: $N_N, \tau_{\text{ftol}}, \tau_{\text{xtol}}, \tau_{\text{stag}}, c_1, \gamma, \alpha_{\min}, N_{\text{ls}}, F_{\text{div}}, N_{\text{inc}}$.

Input: $\mathbf{x}^0 \in \mathcal{X}_h$, nonlinear operator op .

Output: $(\mathbf{x}^*, S, I, \|\mathbf{b}^*\|_\infty, r)$.

```

b  $\leftarrow \mathbf{b}(\mathbf{x}^0)$ ; if  $\|\mathbf{b}\|_\infty \leq \tau_{\text{ftol}}$  then
  | return  $(\mathbf{x}^0, \text{True}, 0, \|\mathbf{b}\|_\infty, \text{InitialFtol})$ ;
end
 $\mathbf{x}_{\text{best}} \leftarrow \mathbf{x}^0$ ;  $b_{\text{best}} \leftarrow \|\mathbf{b}\|_\infty$ ;  $I_{\text{up}} \leftarrow 0$ ;
for  $i = 1, \dots, N_N$  do
  |  $\mathcal{J} \leftarrow \mathcal{J}(\mathbf{x})$ ; recompute weights  $\mathbf{w}$  from  $\text{diag}(\mathcal{J})$  as in (2);  $\Phi \leftarrow \Phi_{\mathbf{w}}(\mathbf{x})$ ;
  |  $(S_{\text{LS}}^{\text{lin}}, \delta\mathbf{x}) \leftarrow \text{SolveLinearSystem}(\mathcal{J}, \mathbf{b})$ ; // Alg. 4
  | if  $S_{\text{LS}}^{\text{lin}} = \text{Failed}$  then
  | |  $\mathbf{x} \leftarrow \mathbf{x}_{\text{best}}$ ; return  $(\mathbf{x}, \text{False}, i, b_{\text{best}}, \text{LinearSolveNaN})$ ;
  | end
  | set  $D \leftarrow -2\Phi$  via the identity (4) (used by the Newton branch of A.2 only);
  |  $(S_{\text{LS}}, \alpha, L, \hat{b}, \hat{\Phi}) \leftarrow \text{ArmijoLinesearch}(\mathbf{x}, \mathbf{x}_{\text{old}}, \mathbf{b}, \delta\mathbf{x}, D, \mathbf{w}, \Phi, \mathbf{m})$ ; // Alg. 5,  $\mathbf{m}$  from  $\text{op}$ 
  | if  $\hat{b} < b_{\text{best}}$  then
  | |  $\mathbf{x}_{\text{best}} \leftarrow \mathbf{x}$ ;  $b_{\text{best}} \leftarrow \hat{b}$ ;
  | end
  |  $(r, S_{\text{conv}}, S_{\text{brk}}, I_{\text{up}}) \leftarrow \text{EvaluateSafeguards}(i, S_{\text{LS}}, L, \hat{b}, \hat{\Phi}, \Phi, I_{\text{up}})$ ; // Alg. 6
  | if  $S_{\text{brk}}$  then
  | | if  $\neg S_{\text{conv}}$  and  $r \notin \{\text{Ftol}, \text{NoiseFloor}, \text{Xtol}\}$  then
  | | |  $\mathbf{x} \leftarrow \mathbf{x}_{\text{best}}$ ;
  | | | end
  | | return  $(\mathbf{x}, S_{\text{conv}}, i, \|\mathbf{b}^*\|_\infty, r)$ ;
  | end
end
return  $(\mathbf{x}, \text{False}, N_N, \|\mathbf{b}\|_\infty, \text{IterationCap})$ ;

```

Reading the box. Each outer iteration computes the Jacobian, delegates the linear solve to Algorithm 4, performs an Armijo backtracking line search via Algorithm 5, and asks Algorithm 6 whether to break, continue, or declare convergence.

Best-state retention. Whenever the residual strictly improves, $(\mathbf{x}_{\text{best}}, b_{\text{best}})$ is updated. On most non-converged exits the iterate is restored to this best-seen point. Three exits intentionally do not restore: Ftol and NoiseFloor are already at or below the target, and Xtol stagnation reflects a state the iterate could not move beyond—restoring would discard the most current information.

Mode independence. The same loop is reused with either an exact-Newton or a Picard Jacobian. The merit function, line search, and safeguards apply unchanged; only the operator op differs. This is what allows Algorithm 2 to compose three calls to this single kernel.

7.2 Algorithm A.1: SolveLinearSystem

Called by: Algorithm 3 (once per Newton iteration)
Calls: Configured linear solver (LU by default)
Inputs: Jacobian $\mathcal{J}$; residual $\mathbf{b}$
Outputs: Failure flag S_{Failed} ; step $\delta\mathbf{x}$

Algorithm 4: SolveLinearSystem: factor and solve, with NaN guard.

Parameters: (none).

Input: $\mathcal{J}$, $\mathbf{b}$.

Output: $(S_{\text{Failed}}, \delta\mathbf{x})$.

```

try { Solve  $\mathcal{J}\delta\mathbf{x} = \mathbf{b}$  via the configured linear solver } catch { return (True,  $\delta\mathbf{x}$ ) ;
  // linear-solver exception
}
if any element of  $\delta\mathbf{x}$  is NaN then
  | return (True,  $\delta\mathbf{x}$ ) ; // catastrophic algebraic failure
end
return (False,  $\delta\mathbf{x}$ );

```

The linear solve is the cheapest place to detect a near-singular Jacobian: either the factorisation aborts with an exception or it returns a vector contaminated by NaNs. Either way the caller restores $\mathbf{x}_{\text{best}}$ and exits.

7.3 Algorithm A.2: ArmijoLinesearch

Called by: Algorithm 3 (once per Newton iteration)

Calls: (none; pure procedure)

Inputs: Current iterate $\mathbf{x}$; reference iterate $\mathbf{x}_{\text{old}}$; residual $\mathbf{b}$; step $\delta\mathbf{x}$; directional derivative $D = -2\Phi$ (cf. §4.2); weights $\mathbf{w}$; base merit Φ ; mode $\mathbf{m} \in \{\text{NEWTON}, \text{PICARD}\}$

Outputs: Acceptance flag S_{LS} ; step size α ; step norm L ; post-step residual $\|\hat{\mathbf{b}}\|_{\infty}$ and merit $\hat{\Phi}$

Algorithm 5: ArmijoLinesearch.

Parameters: $\alpha_{\min}$, c_1 , γ , N_{ls} .

Input: $\mathbf{x}$, $\mathbf{x}_{\text{old}}$, $\mathbf{b}$, $\delta\mathbf{x}$, D , $\mathbf{w}$, Φ , $\mathbf{m}$.

Output: $(S_{\text{LS}}, \alpha, L, \|\hat{\mathbf{b}}\|_{\infty}, \hat{\Phi})$.

```

 $\alpha \leftarrow 1$ ;  $S_{\text{LS}} \leftarrow \text{False}$ ;
for  $j = 1, \dots, N_{\text{ls}}$  do
   $\mathbf{x} \leftarrow \mathbf{x}_{\text{old}} - \alpha \delta\mathbf{x}$ ;  $\hat{\mathbf{b}} \leftarrow \mathbf{b}(\mathbf{x})$ ;  $\hat{\Phi} \leftarrow \Phi_{\mathbf{w}}(\hat{\mathbf{b}})$ ;
  if  $\hat{\Phi}$  or  $\|\hat{\mathbf{b}}\|_{\infty}$  is NaN then
    |  $\alpha \leftarrow \gamma\alpha$ ; continue ; // shrink and retry without testing acceptance
  end
  if  $\mathbf{m} = \text{NEWTON}$  then
    | if  $\hat{\Phi} \leq \Phi + c_1\alpha D$  then
      | |  $S_{\text{LS}} \leftarrow \text{True}$ ; break ; // merit-Armijo accept
    | end
  end
  else //  $\mathbf{m} = \text{PICARD}$ 
    | if  $\|\hat{\mathbf{b}}\|_{\infty} \leq (1 - c_1\alpha)\|\mathbf{b}\|_{\infty}$  then
      | |  $S_{\text{LS}} \leftarrow \text{True}$ ; break ; // monotone-residual accept
    | end
  end
  if  $\alpha \leq \alpha_{\min}$  then
    | break ; // return  $S_{\text{LS}} = \text{False}$ 
  end
   $\alpha \leftarrow \gamma\alpha$ ;
end
 $L \leftarrow \alpha\|\delta\mathbf{x}\|_{\infty}$ ;
return  $(S_{\text{LS}}, \alpha, L, \|\hat{\mathbf{b}}\|_{\infty}, \hat{\Phi})$ ;

```

Mode branching. The Newton branch implements the standard sufficient-decrease test on the equilibrated merit (§4.2); the Picard branch uses a monotone residual reduction in ℓ^{∞} . The acceptance criterion is mode-dependent because the $D = -2\Phi$ identity (eq. (4)) relies on the search direction being

the exact Newton step. Picard steps drop the linearised convective cross-term and the Forchheimer derivative from $\mathcal{J}$, so the identity is generically violated; the merit-based predicate would then be miscalibrated. Replacing it with a contraction test on the ℓ^∞ residual decouples Picard's accept/reject loop from Φ entirely, recovering a sound globalisation step for the fixed-point map. The condition $\|\hat{\mathbf{b}}\|_\infty \leq (1 - c_1\alpha)\|\mathbf{b}\|_\infty$ is the standard Bank–Rose monotone-residual test for damped Picard.

NaN handling. When the residual evaluation at the trial point produces a NaN merit *or* a NaN ℓ^∞ residual norm (typically because the Forchheimer factor $|\mathbf{u}|$ blew up under a too-aggressive step), the line search shrinks α without testing acceptance. The check on $\|\hat{\mathbf{b}}\|_\infty$ is required because the Picard branch never reads $\hat{\Phi}$, and a non-finite residual norm could otherwise reach the contraction test. A Newton step pushing $\mathbf{u}$ into a region where the model momentarily becomes non-evaluable should not be recorded as a failed acceptance step; it should simply be retried at a smaller size.

7.4 Algorithm A.3: EvaluateSafeguards

Called by: Algorithm 3 (once per Newton iteration, after the line search)
Calls: (none; pure decision procedure)
Inputs: Outer iteration index i ; line-search success S_{LS} ; step norm L ; post-step residual $\|\hat{\mathbf{b}}\|_\infty$ and merit $\hat{\Phi}$; base merit Φ ; increase counter I_{up}
Outputs: Exit reason r ; convergence flag S_{conv} ; break flag S_{brk} ; updated counter I'_{up}

Algorithm 6: EvaluateSafeguards: termination arbiter.

Parameters: τ_{ftol} , τ_{stag} , τ_{xtol} , F_{div} , N_N , N_{inc} .

Input: $i, S_{LS}, L, \|\hat{\mathbf{b}}\|_\infty, \hat{\Phi}, \Phi, I_{up}$.

Output: $(r, S_{conv}, S_{brk}, I'_{up})$.

```

if  $\|\hat{\mathbf{b}}\|_\infty \leq \tau_{ftol}$  then
  | return (Ftol, True, True,  $I_{up}$ ) ;                                // converged on residual
end
if  $\neg S_{LS}$  then
  | if  $\|\hat{\mathbf{b}}\|_\infty \leq \tau_{stag}$  then
  | | return (NoiseFloor, True, True,  $I_{up}$ );
  | end
  | return (LinesearchFailed, False, True,  $I_{up}$ );
end
if  $\hat{\Phi} > F_{div}\Phi$  then
  |  $I_{up} \leftarrow I_{up} + 1$ ; if  $I_{up} \geq N_{inc}$  then
  | | return (MeritDivergence, False, True,  $I_{up}$ );
  | end
end
else
  |  $I_{up} \leftarrow 0$ ;
end
if  $L \leq \tau_{xtol}$  then
  | if  $\|\hat{\mathbf{b}}\|_\infty \leq \tau_{stag}$  then
  | | return (NoiseFloor, True, True,  $I_{up}$ );
  | end
  | return (Xtol, False, True,  $I_{up}$ );
end
if  $i = N_N$  then
  | if  $\|\hat{\mathbf{b}}\|_\infty \leq \tau_{stag}$  then
  | | return (NoiseFloor, True, True,  $I_{up}$ );
  | end
  | return (IterationCap, False, True,  $I_{up}$ );
end
return (InProgress, False, False,  $I_{up}$ );

```

In the implementation I_{up} is carried in the `inc_count` field of a mutable `SafeIterationState` struct that the routine updates in place; the box treats it as a returned value for mathematical clarity.

Why six exit reasons? The arbiter distinguishes states that classical Newton implementations conflate, because the choice of reason changes what the caller does next.

- `Ftol` and `InitialFtol` are unambiguous successes.
- `NoiseFloor` is a *conditional* success: the residual sits below τ_{stag} , which is far above floating-point noise but well below physical interest. The caller treats this as success intentionally (§9).
- `LinesearchFailed` and `MeritDivergence` both trigger the Picard fallback (Algorithm 2).
- `Xtol` flags an iterate that has stopped moving even though the residual is non-trivial; failure unless the residual has slipped below τ_{stag} (in which case it is silently relabelled as `NoiseFloor`).
- `IterationCap` is the budget-exhausted failure.
- `LinearSolveNaN` aborts the kernel unconditionally.

The role of F_{div} and N_{inc} together. A single merit-expansion event is not a divergence: Newton near a saddle can take one or two steps that increase the merit before the quadratic basin captures the iterate. Requiring N_{inc} *consecutive* expansions, each by at least a factor F_{div} , separates transient overshooting from genuine repeated divergence.

8 Stage II: Algorithm C (OSGSFractionalRelaxation)

The orthogonal sub-grid scale formulation augments the discrete weak form with a stabilisation term that depends on the L^2 projection π_h of the strong residual. To avoid carrying the Fréchet derivative of Π_h in the global Jacobian, Algorithm 7 treats the pair (U_h, π_h) by block-coordinate relaxation: freeze π_h , solve the inner nonlinear primary system for U_h via the Newton kernel (Algorithm 3), then re-project to get the next π_h .

Called by: Algorithm 1 (Stage II, OSGS mode only)
Calls: Algorithm 2 (once per outer iteration, with π_h frozen; the cascade itself invokes Alg. 3 up to three times)
Inputs: Stage I state U_h^{ASGS} ; pre-factored mass solvers (M_u^{-1}, M_p^{-1}) ; optional block-wise Anderson accelerators
Outputs: Success flag S ; converged projection pair $(\pi_{h,u}, \pi_{h,p})$

Algorithm 7: OSGSFractionalRelaxation: staggered fixed point on (U_h, π_h) .

Parameters: $N_{\text{OSGS}}, N_{\text{inner}}, m_w, \tau_w, \tau_{\text{ftol}}, \tau_{\text{OSGS}}, \tau_{\text{proj}}, \tau_{\text{stag}}, \text{Mode}_{\text{drift}}, \text{Mode}_{\text{stop}}$.

Input: U_h^{ASGS} ; residual operators $(\mathcal{R}_u, \mathcal{R}_p)$; mass-matrix solvers; optional accelerators.

Output: $(S, \pi_{h,u}, \pi_{h,p})$.

```

 $U_h^0 \leftarrow U_h^{\text{ASGS}}; \quad \pi_{h,u}^0 \leftarrow 0; \quad \pi_{h,p}^0 \leftarrow 0; \quad // \text{ see } \S 8.4$ 
for  $m = 1, \dots, N_{\text{OSGS}}$  do
    set effective inner tolerance  $\tau_{\text{ftol},m}^{\text{inner}} \leftarrow (m \leq m_w) ? \max(\tau_{\text{ftol}}, \tau_w) : \tau_{\text{ftol}};$ 
    fix  $\pi_h \leftarrow \pi_h^{m-1}; \quad \mathbf{x}_{\text{prev}} \leftarrow U_h^{m-1}; \quad // \text{ restore vector for the inner cascade}$ 
     $(S_m^{\text{inner}}, U_h^m) \leftarrow \text{RobustNonlinearCascade}(U_h^{m-1}, \mathbf{x}_{\text{prev}}, \text{op}_N^{\text{OSGS}}(\pi_h), \text{op}_P^{\text{OSGS}}(\pi_h));$ 
     $// \text{ Alg. 2; inner cap } N_{\text{inner}}, \text{ tolerance } \tau_{\text{ftol},m}^{\text{inner}}$ 
    if  $S_m^{\text{inner}} = \text{False}$  and the cascade aborted with a structural failure then
        return  $(\text{False}, \pi_{h,u}^{m-1}, \pi_{h,p}^{m-1}); \quad // \text{ see } \S 8.7$ 
    end
     $d_{\text{state}}^m \leftarrow \text{StateDrift}(U_h^m, U_h^{m-1}; \text{Mode}_{\text{drift}}); \quad // \text{ see } \S 8.1$ 
    re-project:  $\pi_{h,u}^m \leftarrow \Pi_{V_{\text{free}}}(\mathcal{R}_u(U_h^m)); \quad \pi_{h,p}^m \leftarrow \Pi_{Q_{\text{free}}}(\mathcal{R}_p(U_h^m));$ 
    if accelerator present then
         $\pi_{h,u}^m \leftarrow \text{Anderson}_u(\pi_{h,u}^{m-1}, \pi_{h,u}^m); \quad // \text{ safety fallback } (\S 8.6) \text{ handled internally}$ 
         $\pi_{h,p}^m \leftarrow \text{Anderson}_p(\pi_{h,p}^{m-1}, \pi_{h,p}^m); \quad // \text{ idem}$ 
    end
     $d_{\pi}^m \leftarrow \max(\|\Delta \pi_{h,u}^m\|_{M_u}, \|\Delta \pi_{h,p}^m\|_{M_p}); \quad // \text{ see } \S 8.2$ 
     $\tau_{\text{OSGS}}^{\text{eff}} \leftarrow \max(\tau_{\text{OSGS}}, \tau_{\text{stag}}); \quad \tau_{\text{proj}}^{\text{eff}} \leftarrow \max(\tau_{\text{proj}}, \tau_{\text{stag}});$ 
    decide  $S_{\text{conv}}$  from  $\text{Mode}_{\text{stop}}, d_{\text{state}}^m, d_{\pi}^m$  (see §8.3);
    if  $m \geq 2$  and  $\|\mathbf{X}_h^m - \mathbf{X}_h^{m-1}\|_{\ell^\infty} \leq \tau_{\text{stag}}$  then
         $S_{\text{conv}} \leftarrow \text{True}; \quad // \text{ absolute noise-floor short circuit}$ 
    end
    if  $S_{\text{conv}}$  then
        if mms_cfg.enabled and first  $S_{\text{conv}}$  event then
            initialise plateau-history  $\mathcal{H}$  at  $U_h^m$ ;  $I_{\text{pass}} \leftarrow 0$ ;
            continue;  $// \text{ do not exit yet; collect a plateau sample}$ 
        end
        if mms_cfg.enabled then
             $(S_{\text{plat}}, I_{\text{pass}}) \leftarrow \text{VerifyMMSPlateau}(U_h^m, U_{\text{ex}}, \mathcal{H}, I_{\text{pass}}); \quad // \text{ Alg. 8}$ 
            if  $\neg S_{\text{plat}}$  then
                continue;
            end
        end
        return  $(\text{True}, \pi_{h,u}^m, \pi_{h,p}^m);$ 
    end
end
return  $(\text{False}, \pi_{h,u}^{N_{\text{OSGS}}}, \pi_{h,p}^{N_{\text{OSGS}}});$ 

```

8.1 State drift: ℓ^∞ vs continuous L^2

The state drift between consecutive outer iterations

$$d_{\text{state}}^m = \begin{cases} \max(\|\mathbf{u}_h^m - \mathbf{u}_h^{m-1}\|_{L^2(\Omega)}, \|\mathbf{p}_h^m - \mathbf{p}_h^{m-1}\|_{L^2(\Omega)}) & \text{Mode}_{\text{drift}} = \text{L2_mass}, \\ \|\mathbf{X}_h^m - \mathbf{X}_h^{m-1}\|_{\ell^\infty} & \text{Mode}_{\text{drift}} = \text{Linf}, \end{cases} \quad (5)$$

admits two metrics with subtly different meanings. The `Linf` mode is the raw vector-space ℓ^∞ norm of the concatenated free-DOF array; it is cheap but pathological in mixed-physics systems because it lets the field with larger numerical magnitude dominate. The `L2_mass` mode computes two independent continuous $L^2(\Omega)$ norms, $\sqrt{\int_{\Omega} (\mathbf{u}_h^m - \mathbf{u}_h^{m-1}) \cdot (\mathbf{u}_h^m - \mathbf{u}_h^{m-1}) \, d\Omega}$ and analogously for the pressure block, evaluated by Gridap quadrature on the same measure $d\Omega$ used by the formulation, and takes their maximum. The distinction matters: `L2_mass` is the continuous functional norm of the difference FE function, whereas

`Linf` is the discrete maximum coefficient deviation, which is basis-dependent and mixes physical units across blocks. Only `L2.mass` measures velocity and pressure in their own natural functional norms.

8.2 Projection drift in the mass-weighted inner product

The projection drift

$$d_\pi^m = \max\left(\sqrt{(\Delta\pi_{h,u}^m)^\top M_u (\Delta\pi_{h,u}^m)}, \sqrt{(\Delta\pi_{h,p}^m)^\top M_p (\Delta\pi_{h,p}^m)}\right) \quad (6)$$

is the natural distance in the inner product that defines the L^2 projection itself. The Euclidean DOF norm would conflate this with inverse-mass scaling and report misleading drift values on graded meshes.

8.3 Choosing the stopping mode

The user selects `Mode_stop` between three options:

- `state_drift`: terminate when $d_{\text{state}}^m \leq \tau_{\text{OSGS}}^{\text{eff}}$.
- `projection_drift`: terminate when $d_\pi^m \leq \tau_{\text{proj}}^{\text{eff}}$.
- `both` (default): require both predicates.

`both` is the strictest and corresponds to the mathematical definition of the staggered fixed point. The single-criterion modes sacrifice one of the two conditions in exchange for fewer iterations and are appropriate only when the user knows that the slacker condition is already satisfied. In all three cases an additional absolute short-circuit fires when the raw ℓ^∞ DOF drift drops below τ_{stag} for $m \geq 2$; this protects against limit-cycle oscillations that never cleanly satisfy τ_{OSGS} or τ_{proj} .

8.4 Bootstrap, mass-matrix pre-factorizing, and warmup

Zero bootstrap of π_h^0 . The first sub-grid projection is initialised to zero, $\pi_h^0 \equiv 0$, *not* to the L^2 projection of the strong residual at U_h^{ASGS} . The choice is deliberate. Solving the OSGS inner problem at $m = 1$ with $\pi_h \equiv 0$ is, by definition, exactly the ASGS problem, and Stage I has already converged that system to within τ_{ftol} . The very first inner solve therefore starts and ends at U_h^{ASGS} , and the first non-trivial projection π_h^1 is produced naturally by the L^2 -projection step at the bottom of the outer-loop body. Pre-seeding π_h^0 with $\Pi(\mathcal{R}(U_h^{\text{ASGS}}))$ would instead drive the $m = 1$ inner solve away from U_h^{ASGS} to satisfy a stabilisation term whose projection has not yet been relaxed by an outer iteration—an “off-distribution” bootstrap that historically required the warmup phase (see below) to unwind.

Mass-matrix pre-factorizing. Each projection is the solution of an SPD linear system with M_u or M_p . Because the projection target spaces never change between outer iterations, the matrices and their LU factorisations are assembled once (in Algorithm 1) and reused: every projection is a constant-cost forward/back substitution.

Warmup phase. The warmup phase ($m \leq m_w$) relaxes the inner-cascade tolerance to $\max(\tau_{\text{ftol}}, \tau_w)$. Even with the zero bootstrap of π_h^0 , the projection π_h^m for small m is still moving rapidly between outer iterations; driving the inner solve to a tight τ_{ftol} in this regime would polish the iterate against a projection that gets replaced immediately. Relaxing the inner tolerance during the warmup trades a small amount of accuracy in the early iterates for a meaningful cut in inner-solver budget.

8.5 Projection policy and the constant- σ trim

The projection step $\pi_h^m \leftarrow \Pi(\mathcal{R}(U_h^m))$ does not always project the full strong residual; the policy is configurable, and the choice is determined jointly by the discrete formulation and the reaction model.

The default policy projects the full residual. When the reaction tensor $\sigma(\alpha, \mathbf{u})$ is a constant (`reaction_model = "Constant"` in `stabilization_config`, paired with the `ProjectResidualWithoutReactionWhenConstantSigma` policy), the reaction term $\sigma \mathbf{u}_h$ is omitted from the residual before projection. The justification is exactness: when σ is a spatial constant, $\sigma \mathbf{u}_h$ lies entirely in the FE velocity space $\mathbf{V}_h$ (which is closed under multiplication by a scalar), so its L^2 projection onto V_{free} is the function itself; subtracting it again in the orthogonal complement $I - \Pi_h$ gives exactly zero. Including the reaction term in the projection step would amount

to projecting and re-subtracting an FE function—a no-op in exact arithmetic, but one that introduces avoidable floating-point noise. Removing it from the projected residual eliminates that noise at no cost. This corresponds to Section 4.4 of the parent paper.

For non-constant σ (e.g. the Forchheimer–Ergun law), the reaction term is *not* an FE function and its projection is non-trivial; the trim does not apply and the policy reverts to the default of projecting the full residual.

8.6 Block-wise Anderson acceleration

For stiff fixed points on π_h , Anderson acceleration extrapolates a new iterate from the last m fixed-point residuals by solving a small mass-weighted least-squares problem

$$\min_{\gamma} \|(\Delta F)\gamma - f_k\|_M^2. \quad (7)$$

The accelerator operates independently on $\pi_{h,u}$ and $\pi_{h,p}$, each with its own mass matrix; combining the blocks would force a cross-physical mixing that the OSGS formulation goes out of its way to avoid. The least-squares system uses the same L^2 inner product that defines the underlying projection. A safety fallback discards the extrapolation and reverts to a relaxed Picard step when the extrapolated state lies more than a fixed multiple of the unextrapolated residual away from the raw fixed-point candidate.

8.7 Failure of the inner cascade

The inner solve inside Algorithm 7 is the same RobustNonlinearCascade used in Stage I, supplied with OSGS operators (so the Jacobian depends on the frozen projection π_h^{m-1}) and with the iteration cap reduced to N_{inner} on each inner ExactNewtonPipeline call. The restore vector is $\mathbf{x}_{\text{prev}} = U_h^{m-1}$, i.e. the iterate at the start of the current outer iteration; this differs from Stage I, which restores to the global initial guess $\mathbf{x}_{\text{backup}}^0$.

Three failure modes are propagated as structural divergence and abort the outer loop:

- The first exact-Newton attempt produces a non-finite iterate *and* the Picard attempt subsequently produces a non-finite iterate as well (the cascade then returns $\mathbf{x}_{\text{restore}}$ with $S = \text{False}$).
- The Picard attempt itself raises an unrecoverable exception (distinct from the benign “reached maximum iterations” exception that is reinterpreted as a completed Picard smoothing pass).
- The terminal Newton retry, after a Picard smoothing pass, returns with one of `LinesearchFailed`, `MeritDivergence`, or `LinearSolveNaN`.

All other exit configurations (notably `IterationCap` from exhausting N_{inner} inside one of the cascade’s three sub-calls) are treated as progress: the next outer iteration simply re-engages with the updated projection.

9 Convergence-Analysis Notes

9.1 The false-equivalency failsafe

The single most important invariant of Algorithm 6 is the asymmetry between two exit reasons that, at first glance, look interchangeable:

- `IterationCap` with $\|\mathbf{b}\|_{\infty} \gg \tau_{\text{stag}}$: the iteration ran out of budget while still far from any noise floor.
- `NoiseFloor` with $\|\mathbf{b}\|_{\infty} \leq \tau_{\text{stag}}$: the iteration may have stopped for any reason, but the residual is below the noise floor we set for “effectively converged”.

The orchestration treats only the second as success. Coercing the first into a converged state—“it ran for many iterations and stopped; close enough”—is forbidden, because doing so would feed an unresolved iterate into Stage II of the cascade. In OSGS in particular, an unresolved U_h produces a stale π_h whose subsequent fixed-point iterations cannot recover.

9.2 Why the OSGS stopping mode matters

The default `both` predicate corresponds to the mathematical definition of the staggered fixed point: (U_h, π_h) is the converged pair when neither moves between outer iterations. The relaxed modes are appropriate only when the user knows that the slacker condition is already satisfied; using a single-criterion mode without that knowledge introduces the false-convergence trap that the noise-floor invariant forbids.

A Code–Notation Correspondence

Table 3 maps every exit-reason label used by the algorithm boxes to the string literal returned by the implementation.

Table 3: Exit reasons in Algorithm 6 and their code-side strings.

Box label	Code string	Meaning
Ftol	"ftol_reached"	$\ b\ _\infty \leq \tau_{\text{ftol}}$ after a successful line search.
InitialFtol	"initial_ftol"	Initial guess already satisfies τ_{ftol} ; no Newton step taken.
NoiseFloor	"stagnation_noise_floor_reached"	$\ b\ _\infty \leq \tau_{\text{stag}}$ on a non-converged exit.
LinesearchFailed	"linesearch_failed"	Line-search step shrank to $\alpha \leq \alpha_{\min}$ without descent.
MeritDivergence	"merit_divergence_escaped"	Merit function expanded by $\geq F_{\text{div}}$ for N_{inc} consecutive steps.
Xtol	"xtol_stagnation"	Step norm $L \leq \tau_{\text{xtol}}$ with residual still above τ_{stag} .
IterationCap	"max_iters_stagnation"	Iteration budget exhausted with residual still above τ_{stag} .
LinearSolveNaN	"linear_solve_nan"	Linear-solve exception or NaN in δx .
InProgress	" "	Sentinel for “no break this iteration”.

Part II

Optional Extensions

The remainder of this document describes *optional extensions* to the core algorithm of Part I. These extensions are not part of a generic `run_simulation` call: each is activated by a specific configuration setting or a specific calling context (a parameter sweep, a verification harness, etc.). Where an extension augments a core algorithm box, the additional lines are shown in this colour. Removing every coloured line from this part yields the core algorithm of Part I, byte-for-byte.

B Extension Hierarchy

Two extensions are documented. The first (§C) adds an MMS plateau verifier (Algorithm 8) that hooks into the orchestrator (Algorithm 1) and the OSGS loop (Algorithm 7) by inserting coloured lines directly in those algorithm boxes (§C.4). The second (§D) is a set of parameter substitutions applied by the sweep harness; it does not introduce new algorithm boxes. Figure 2 overlays the extension onto the core tree of Figure 1.

C Method of Manufactured Solutions Plateau Verifier

C.1 When this extension fires

This extension is active whenever the solver is supplied with a configuration `mms_cfg` whose `enabled` flag is true. The `mms_cfg` object carries an analytical reference solution $U_{\text{exact}} = (u_{\text{ex}}, p_{\text{ex}})$ (via an `oracle` callback) and a small set of plateau-test parameters. When `mms_cfg` is absent or disabled, the core orchestration runs unchanged.

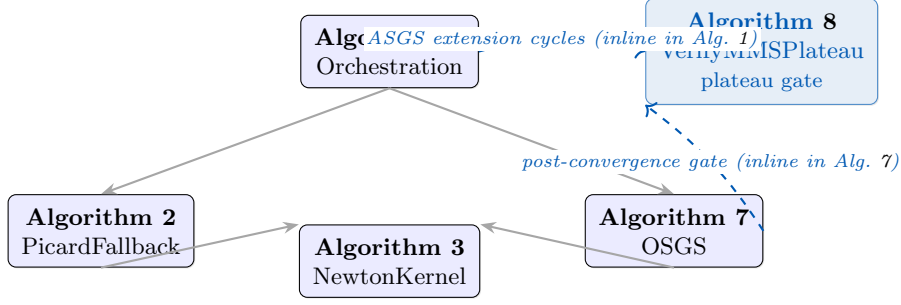


Figure 2: Extension hierarchy. The MMS plateau verifier (Algorithm 8, blue) is hooked directly into the core orchestrator (Algorithm 1) and the core OSGS loop (Algorithm 7) by inline `\extline{}` blue lines, displayed within the respective core algorithm boxes (§C.4). Solid arrows reproduce the core hierarchy of Figure 1; *dashed coloured arrows* mark the extension hooks.

C.2 Numerical versus discretisation convergence

The inner Newton stops when $\|\mathbf{b}\|_\infty \leq \tau_{\text{ftol}}$. That is convergence in the *algebraic* sense: the iterate is close to the root of the discrete system. It says nothing about whether the discrete system is close to the true PDE solution. The Method of Manufactured Solutions (MMS) provides exactly that second test: it injects an exact analytical $(\mathbf{u}_{\text{ex}}, p_{\text{ex}})$, recomputes the appropriate source terms, and measures $\|U_h - U_{\text{ex}}\|$ in physical norms. The expected discretisation behaviour is $O(h^{k+1})$ in the L^2 norms (one order weaker in the H^1 -semi norm).

For a single mesh, plateau verification asks a different question: once the iterate is close enough to the root that further iterations do not move the FE error, the FE error has *plateaued* at its discretisation floor.

C.3 Algorithm D and its interface

Algorithm 8 is itself a leaf—a single side-by-side comparison of consecutive FE errors against the exact solution. It does not advance the iterate.

Called by: Inline blue hook in Algorithm 1 (per extension cycle in the ASGS path); inline blue hook in Algorithm 7 (per OSGS staggered iteration once S_{conv} has fired). See §C.4.
Calls: (none; pure procedure)
Inputs: Current iterate U_h^m ; exact reference U_{ex} ; running error history $\mathcal{H}$; pass counter I_{pass}
Outputs: Plateau flag S_{plat} ; updated counter I'_{pass}

Algorithm 8: VerifyMMSPlateau: relative-change plateau test (optional extension).

Parameters: $N_{\text{plat}}, \tau_{\text{err}}, \varepsilon_{u,L^2}, \varepsilon_{u,H^1}, \varepsilon_{p,L^2}$.

Input: $U_h^m, U_{\text{ex}}, \mathcal{H}, I_{\text{pass}}$.

Output: $(S_{\text{plat}}, I'_{\text{pass}})$.

```

compute  $E_{u,L^2}^m, E_{u,H^1}^m, E_{p,L^2}^m$ ;
append to  $\mathcal{H}$ ; let  $E_{\bullet}^{m-1}$  be the previous entry;
 $r_{\bullet} \leftarrow |E_{\bullet}^m - E_{\bullet}^{m-1}| / \max(E_{\bullet}^m, E_{\bullet}^{m-1}, \varepsilon_{\bullet})$  for each norm;
 $r_{\text{max}} \leftarrow \max_{\bullet} r_{\bullet}$ ;
if  $r_{\text{max}} \leq \tau_{\text{err}}$  then
  |  $I_{\text{pass}} \leftarrow I_{\text{pass}} + 1$ ;
end
else
  |  $I_{\text{pass}} \leftarrow 0$ ;
end
if  $I_{\text{pass}} \geq N_{\text{plat}}$  then
  | return (True,  $I_{\text{pass}}$ ) ; // plateau satisfied
end
return (False,  $I_{\text{pass}}$ ) ; // continue extending

```

The verifier tracks three error quantities, not two: L^2 of velocity, L^2 of pressure, and the H^1 -semi-

norm of velocity. The third matters because the L^2 errors can plateau while the velocity gradient—which is what the convective and viscous terms actually feel—still drifts. The plateau is declared only after N_{plat} *consecutive* passes, and the counter resets to zero on the first failure.

C.4 How Algorithm D hooks into the core

The MMS plateau verifier is wired into the core algorithms by two short blocks of conditional logic, displayed inline in Algorithm 1 (Stage II dispatch) and Algorithm 7 (post- S_{conv} branch) and typeset in this colour. The core path is recovered byte-for-byte by deleting every coloured line.

In the ASGS branch of Algorithm 1, when `mms_cfg.enabled` is true, the orchestrator enters a bounded plateau loop: up to K_{extra} extra single-Newton-step cycles are run on top of the converged U_h^{ASGS} , each followed by a call to Algorithm 8. The loop exits as soon as the plateau condition is satisfied or the budget is exhausted.

In the OSGS branch, the augmentation has two parts. The orchestrator inflates the outer-loop budget ($N_{\text{OSGS}}^{\text{eff}} \leftarrow N_{\text{OSGS}} + K_{\text{extra}}$) to leave room for the plateau samples. Inside Algorithm 7, the S_{conv} branch is no longer terminal: the first S_{conv} event is intercepted to seed the plateau history and is followed by a `continue`, so the outer loop runs one more iteration to collect a sample; on subsequent S_{conv} events, Algorithm 8 is called and the loop either exits (plateau satisfied) or continues (still extending).

C.5 Parameters of this extension

Table 4: Numerical parameters of the MMS plateau verifier (extension-only).

Symbol	Field in code	Role / effect
N_{plat}	<code>mms.require_consecutive_passes</code>	Consecutive passes of $r_{\text{max}} \leq \tau_{\text{err}}$ required before declaring a plateau.
τ_{err}	<code>mms.tau_err</code>	Relative-change tolerance against which r_{max} is tested.
$\varepsilon_{u,L^2}, \varepsilon_{u,H^1}, \varepsilon_{p,L^2}$	<code>mms.eps_u_l2, _u_h1, _p_l2</code>	Per-norm floors guarding the denominator in r_{max} .
K_{extra}	<code>mms.max_extra_cycles</code>	Cap on extension cycles (ASGS path) or staggered iterations added to the OSGS budget (OSGS path) for plateau verification.

D Parameter-Sweep Harness Adaptations

D.1 When this extension fires

These adaptations are active only when the solver is driven by the project’s parameter-sweep harness, which wraps the core `run_simulation` pipeline in a triple loop over Re , Da , and the mesh-resolution parameter. In an ad-hoc `run_simulation` call, none of these adaptations fires; the core algorithm of Part I uses the static parameters of Table 1 verbatim. The harness changes parameter *values* only; the algorithm boxes themselves are unchanged, so no O' or B' variants are needed.

D.2 Adaptive τ_{ftol}

Two error scales matter in an FE method. The *algebraic error* is the distance between the iterate and the discrete root, governed by τ_{ftol} . The *discretisation error* is the distance between the discrete root and the true PDE solution, which behaves as $O(h^{k_v+1})$. The harness therefore re-derives τ_{ftol} per case:

$$\tau_{\text{ftol}}^{\text{dyn}} = \max\left(\tau_{\text{ftol}}^{\text{static}}, \min(C_{\text{ceil}}, C_{\text{sf}} \cdot h^{k_v+1})\right). \quad (8)$$

The right-most term ties the tolerance to the discretisation scale via a spatial safety factor C_{sf} ; the ceiling C_{ceil} prevents trivially loose tolerances on coarse meshes; the outer max prevents the formula from descending below the user-supplied $\tau_{\text{ftol}}^{\text{static}}$.

D.3 Adaptive noise floor

Linear-system conditioning grows as $\mathcal{O}(h^{-2})$ for elliptic operators and is amplified by convective and reactive terms. A static τ_{stag} appropriate on a coarse mesh becomes far above the genuine round-off floor on a fine mesh. The harness scales

$$\tau_{\text{stag}}^{\text{dyn}} = \min\left(\tau_{\text{stag}}, \max(n^{\text{min}}, n^{\text{base}} \cdot N^2 \cdot \max(1, \text{Re}))\right), \quad (9)$$

with the additional clamp $\tau_{\text{stag}}^{\text{dyn}} \leftarrow \max(\tau_{\text{stag}}^{\text{dyn}}, n^{\text{sf}} \tau_{\text{ftol}}^{\text{dyn}})$ preserving the strict ordering $\tau_{\text{stag}}^{\text{dyn}} > \tau_{\text{ftol}}^{\text{dyn}}$.

D.4 Per-case Picard widening

In a $(\text{Re}, \text{Da}, h)$ sweep, the harness widens the Picard iteration budget when a stiffness threshold is exceeded:

$$\begin{aligned} N_{\text{P}}^{\text{local}} &\leftarrow N_{\text{P}}, \\ \text{if } \text{Re} \geq \text{Re}^{\text{th}} : \quad &N_{\text{P}}^{\text{local}} \leftarrow \max(N_{\text{P}}^{\text{local}}, N_{\text{P}}^{\text{Re}}), \\ \text{if } \text{Da} \geq \text{Da}^{\text{th}} : \quad &N_{\text{P}}^{\text{local}} \leftarrow \max(N_{\text{P}}^{\text{local}}, N_{\text{P}}^{\text{Da}}). \end{aligned} \quad (10)$$

D.5 Parameters of this extension

Table 5: Numerical parameters of the parameter-sweep harness adaptations (extension-only).

Symbol	Field in code	Role / effect
$C_{\text{ceil}}, C_{\text{sf}}$	<code>sol.dynamic_ftol_ceiling, _spatial_safety_factor</code>	Clamps for the adaptive τ_{ftol} rule, §D.2.
$n^{\text{base}}, n^{\text{min}}, n^{\text{sf}}$	<code>sol.condition_noise_floor_*</code>	Adaptive noise-floor controls, §D.3.
$\text{Re}^{\text{th}}, N_{\text{P}}^{\text{Re}}$	<code>sol.dynamic_picard_re_*</code>	Per-case Picard widening trigger and budget for high Reynolds.
$\text{Da}^{\text{th}}, N_{\text{P}}^{\text{Da}}$	<code>sol.dynamic_picard_da_*</code>	Per-case Picard widening trigger and budget for high Damköhler.